歌曲資料庫系統

學號:1258001　　姓名:江永閎

1.　資料處理流程

- **使用之資料表與欄位說明:**

  系統採用關聯式資料庫設計，核心包含五張資料表：

  I. Songs(歌曲表):儲存作品核心資訊
  - song_id(PK):歌曲唯一識別碼
  - song_title:作品名稱
  - song_language:語系（國語、英語、粵語等）
  - album_id(FK):關聯至專輯
  - play_count:點播計數器（用於排行榜）

  II. Albums(專輯表):儲存專輯集合
  - album_id(PK), album_title, album_date

  III. Creators(創作者表):儲存所有人員資料
  - creator_id(PK), creator_name, creator_country

  IV. Song_Creator_Roles(角色關聯表):解決歌曲與創作者間的「多對多」關係
  - song_id, creator_id, role（歌手、作曲、作詞）

  V. Songs_Deleted_Log(刪除紀錄表):由 SQL Trigger 自動寫入，用於審計追蹤

  說明:

  - 創作者(Creator)用來儲存所有人物的獨立資料，無論他們是歌手、作詞者、作曲者或擁有多重身份，這是人物資訊的唯一來源，創作者可能有一首以上的歌曲。
  - 歌曲與創作者角色關聯表(Song_Creator_Role)是一個關聯表，用來詳細記錄哪位創作者在哪一首歌曲中扮演了什麼角色。
  - 歌曲實體是資料庫的核心，代表單一的音樂曲目。它包含了自身的描述性資訊，並透過屬性關聯到「專輯」實體。
  - 同一首歌只會出現在一張專輯裡，而一張專輯有一首以上的歌曲，每一首歌有一位以上的創作者，每一位創作者也可能有一首以上的歌曲。

- **資料新增、查詢、修改與刪除(CRUD)之操作流程:**
  - 新增(Create):採用 SQL Transaction(交易控制)。在新增歌曲時，系統會先檢查是否需建立新專輯或新創作者，待所有父表資料寫入後，再寫

入歌曲與角色關聯。若中途失敗，則執行 Rollback 確保資料一致性。

- 查詢(Read):系統透過 LEFT JOIN 整合多張表，並使用 STRING_AGG 函數將同一首歌的多位創作者合併為字串顯示。
- 修改(Update):/edit 功能會先更新歌曲主表，接著「刪除並重建」該歌曲的製作群關聯，確保角色變更不會產生殘留資料。此外，KTV 點歌會觸發 play_count 的+1 更新。
- 刪除(Delete):執行刪除時，資料庫內部的 Trigger(觸發程序)會自動將該歌曲標題與時間備份至 Songs_Deleted_Log，實現「刪除即備份」。

- **使用者輸入資料之來源與用途:**
  - 來源:主要來自管理者的「發行錄入表單」與使用者的「KTV 點歌操作」。
  - 用途:輸入的文字資料用於建立索引以便搜尋；選擇的下拉選單用於維護資料的一致性；點擊行為則轉換為行為數據，供排行榜與 AI 模型進行權重計算。
- **系統輸出結果之呈現方式與意義:**
  - 列表呈現:使用 Bootstrap 表格展示，每一列代表一首歌曲。
  - 創作者整合顯示:透過後端 SQL 的聚合函數，將原本分散在多筆資料列的創作者資訊，整合在單一儲存格中顯示。這讓使用者能一眼看出該歌曲的所有參與人員及其角色，體現了資料庫正規化後透過程式邏輯還原資訊的能力。

2. 系統操作流程

- 完整部署與啟動流程
  - 第一步：SQL Server 資料庫建置

I. 開啟 SQL Server Management Studio (SSMS)。

II. 開啟新增查詢執行以下程式碼:

```
USE master;
GO

-- 啟用進階選項（如果尚未啟用）
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
GO

-- 啟用外部腳本功能
-- 參數值 1 代表啟用 (Enabled)
EXEC sp_configure 'external scripts enabled', 1;
RECONFIGURE WITH OVERRIDE;
GO
```

```
PRINT '外部腳本功能已設定為啟用狀態。';
PRINT '請執行步驟 2：重新啟動 SQL Server 服務。';
```

III. 重啟 SSMS，在 SSMS 中右鍵點擊伺服器名稱，選擇停止(Stop)。再次右鍵點擊，選擇啟動(Start)。

IV. 打開 Windows 的服務(Services)應用程式(services.msc)，在裡面找到名為 SQL Server Launchpad 的服務，確認其狀態(Status)為正在執行(Running)，且啟動類型(Startup Type)為自動(Automatic)，如果狀態不是「正在執行」，右鍵點擊並選擇啟動(Start)。

V. 執行 MusicLibraryDB.sql 腳本，自動建立資料庫、資料表、預存程序(SP)、自訂函數 (Function) 以及負責機器學習運算的腳本環境。

◆ 第二步:Python 環境準備(CMD 指令執行)

I. 開啟 CMD 命令提示字元。

II. 切換至專案資料夾路徑。

III. 使用 pip 安裝必要的套件庫：

```
pip install flask pyodbc
```

◆ 第三步：設定資料庫連線串

I. 開啟 1258001_江永閔_網站開發.py 檔案。

II. 確認 DB_CONFIG 中的 SERVER(在第 23 行，目前預設為 localhost)的環境相符，需與先前執行的 SQL 同環境。

◆ 第四步：啟動網站程式

I. 在 CMD 輸入啟動指令：

```
python 1258001_江永閔_網站開發.py
```

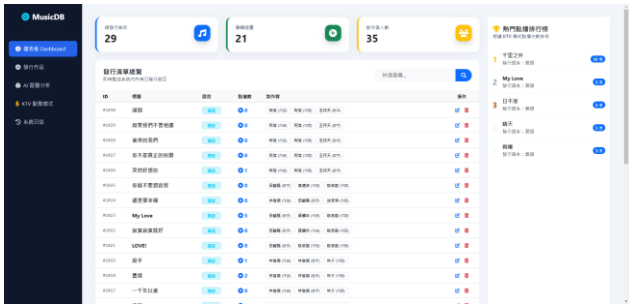
II. 當畫面上顯示 Running on http://127.0.0.1:5000 時，代表 Flask 伺服器已成功啟動。

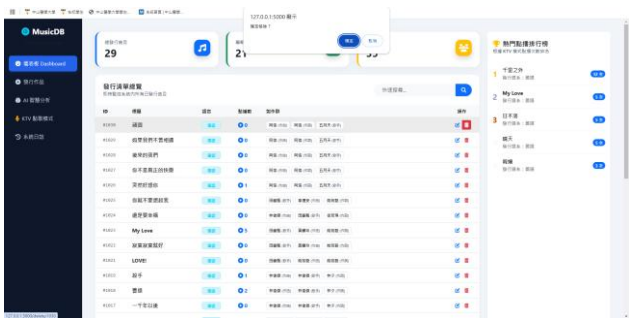
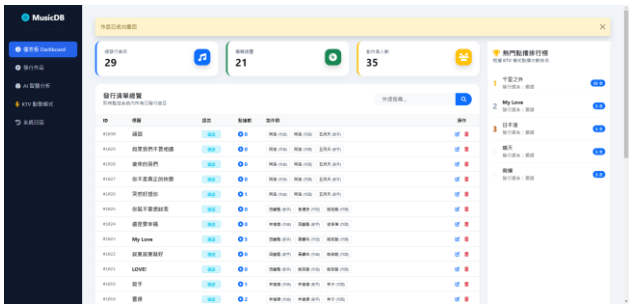
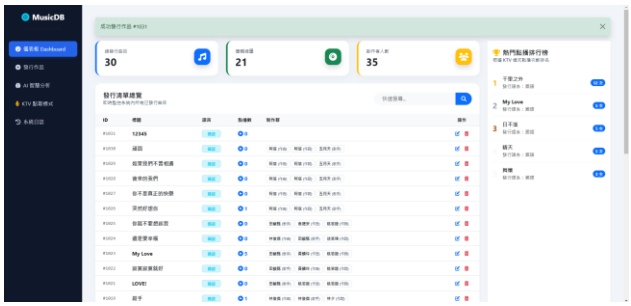
◆ 第五步：進入系統頁面

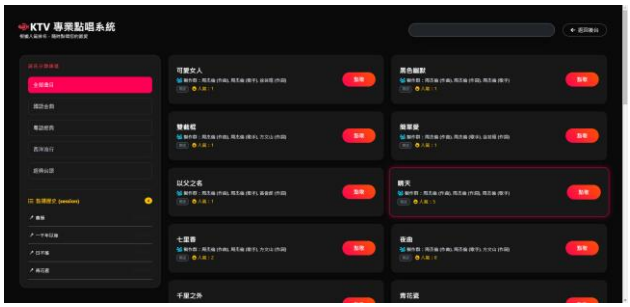
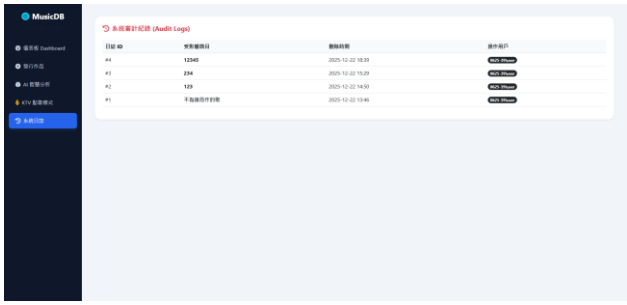
I. 開啟瀏覽器，輸入網址：http://localhost:5000(依實際跳出之網址為主)。

II. 進入後即可看到首頁儀表板，並可透過側邊導覽切換至各功能模組。

■ 功能流程說明&操作畫面截圖&系統回饋結果展示

	首頁提供總覽，包含自動產生的熱門點播排行榜、總發行曲目、專輯總量、創作者人數、發行清單總攬等。

	進入歌曲列表後，點擊「移除」可刪除該曲目，系統會跳出移除確認，避免誤刪。
	歌曲成功編輯、取消編輯、成功移除或取消移除，都會有相對應的通知。
	進入歌曲列表後，點擊「編輯」圖示可修改內容。系統會引導至編輯頁面，管理員可修正語系、重選專輯或更新製作群。
	進入發行作品，可以新增歌曲名稱、語言、所屬專輯及製作群。
	發行作品完成後會直接跳回首頁，並發送發行成功的通知。

	點擊 AI 智慧分析，系統將透過 SQL Server 執行即時機器學習，並以圓餅圖呈現各類人才的分佈比例。
	進入 KTV 點歌模式，使用者可根據歌名或人名搜尋。點擊「點歌」按鈕時，前端會通知後端執行 UPDATE play_count，達成首頁的熱門點播排行榜數據追蹤。
	點選系統日誌，裡面會記錄所有新增與刪除的資訊。

3. 成果檢討

在開發過程中，我們遇到了多個關鍵技術挑戰，例如:外部指令碼縮排問題，由於 SQL 的 sp_execute_external_script 對 Python 代碼的縮排極度敏感，Flask 的多行字串縮排導致 Python 報出 IndentationError，經過重新格式化 SQL 指令，將嵌入的 Python 代碼強制靠左對齊，解決了機器學習跑不動的問題。還有交易提交機制(Transaction)在初期測試發現新增的作品刷新後會自動消失，發現 pyodbc 預設不自動提交交易，於是我在後端加入 conn.commit()，確保每一筆寫入操作都能持久化儲存在硬碟中。以及多對多關聯的複雜性，一首歌同時有歌手、作詞、作曲，這在資料庫中需要處理三筆 Roles 紀錄，因此開發了「一站式錄入介面」，讓管理員在一個表單內完成三個角色的設定，後端再利用循環邏輯自動分配寫入各張關聯表。

然而，由於一開始沒有預設需儲存歌曲的音檔，因此這個資料庫只能存取歌曲相關訊息，無法實際儲存音樂，若需改善此問題，須於初始設計時直接保留能夠存取因檔的位置，以達成更完整的音樂資料庫。